

Fake News & Misinformation Detection System

Final Submission Report

Seminar Project Team

June 17, 2026

Contents

1 Executive Summary

This project delivers an end-to-end natural language processing system for fake news and misinformation detection with production-oriented design choices rather than a notebook-only prototype. The final system combines a classical baseline, a transformer classifier, an agentic orchestration layer, evidence retrieval, explainability, privacy protection, monitoring, and a deployable user interface. The implementation directly addresses the rubric in Prokct-2 by covering business framing, data handling, model development, evaluation, deployment, responsible AI, and agentic reasoning.

The deployed inference stack exposes a FastAPI backend and a Streamlit frontend. The backend provides prediction, analysis, health, version, and agent endpoints; the frontend renders predictions, evidence cards, trust signals, and execution traces. The core classifier is a fine-tuned RoBERTa model, while a TF-IDF plus Logistic Regression baseline serves as a reproducible comparison point. The system also includes Optuna-based hyperparameter tuning, privacy masking before inference, drift and confidence monitoring, and a dedicated UNCERTAIN review state for evidence-conflict or low-confidence cases.

The key engineering goal of the project is not only strong classification performance but also trustworthy behavior in demonstration and operational settings. For that reason, the final design treats model confidence as only one signal among several. External evidence, source credibility, evidence quality, contradiction detection, and trace capture all contribute to the final user-facing result. This allows the system to explain why a claim was labeled REAL, FAKE, or UNCERTAIN and makes the workflow appropriate for academic defense and product-style review.

2 Problem Definition

Fake news detection is a high-value NLP problem because misinformation spreads faster than manual verification processes can scale. Moderation teams, fact-checking groups, and internal trust-and-safety operations need tools that can triage content, reduce review time, and surface signals for human judgment. An automated system has practical value if it can support reviewers with fast, consistent, and explainable recommendations rather than opaque labels.

In this project, the target stakeholders are moderators, newsroom verification staff, and platform operators who need an evidence-aware first-pass assessment of article-like text. The business value comes from reduced manual effort, improved response time, and better prioritization of high-risk content. The technical objective is to build a binary classification system with strong accuracy and F1 while preserving deployment readiness and interpretability. The operational objective is to expose that model in a user-facing system that remains safe under imperfect evidence conditions.

NLP is required because the input is free-form text whose meaning depends on word choice, semantic context, and longer-range dependencies. Traditional heuristics alone are not sufficient for robust classification, especially for ambiguous or adversarially phrased claims. A transformer model such as RoBERTa can capture these linguistic dependencies more effectively than a bag-of-words model, while retrieval and explainability components provide external grounding and human-readable output.

The main success metrics are technical and operational. Technical metrics include accuracy, precision, recall, F1, and response latency. Operationally, the system should provide a usable explanation, retrieve relevant evidence when available, and signal uncertainty when classifier and evidence do not align. Those requirements shaped the final architecture more strongly than

raw benchmark performance alone.

3 Literature Review

Fake news detection has been studied from multiple directions, including lexical feature engineering, social-context modeling, graph methods, and transformer-based classification. Earlier practical systems commonly used TF-IDF features with linear classifiers because those methods are efficient, interpretable, and easy to deploy. They remain important as baselines because they reveal whether a dataset and preprocessing pipeline are already sufficient to solve a simplified version of the task.

Recent progress in transformer language models has shifted the state of practice toward contextual representation learning. BERT demonstrated the value of deep bidirectional pretraining for downstream classification tasks, and RoBERTa improved on that recipe through larger-scale and more robust pretraining choices. These models are especially suitable for misinformation detection because deceptive claims often depend on context, framing, negation, and semantic composition rather than isolated keywords.

A second relevant line of work concerns explanation and trust. High-accuracy models are not enough in domains where a user may question the basis of a prediction. Token-level attribution, evidence retrieval, and conflict detection are increasingly used to make model outputs reviewable. This project follows that direction by combining model prediction with evidence-aware reasoning instead of presenting confidence alone as the final answer.

A third line of work concerns production deployment and continual monitoring. Many academic NLP systems stop at offline evaluation, but production-style systems must consider API behavior, privacy, drift, observability, and model lifecycle. The course rubric explicitly requires those aspects, so this project extends beyond training into deployment architecture, monitoring artifacts, and conceptual retraining strategy.

4 Dataset Description

The repository configuration defines a local dataset source at `data/raw/fakenewsnet.csv` with fields `text` and `label`. The dataset configuration uses binary labels `fake` and `real`, which aligns with both the baseline and transformer classification pipelines. The final system assumes article-level input and binary decision output, with later evidence-aware logic supporting an operational UNCERTAIN review state.

The data configuration specifies a 70/15/15 train, validation, and test split with random seed 42 and stratification enabled. This is implemented in the repository rather than stated only in documentation, which supports reproducibility. Stratification is particularly important in fake news detection because skewed label distributions can inflate naive accuracy while harming minority-class behavior.

The repository includes structured data handling code under `src/data/`, including dataset preparation, preprocessing, validation, and splitting modules. This means data management is not hidden inside notebooks but encoded as reusable software components. That supports the course requirement for professional structure and makes it easier to rerun experiments or audit assumptions.

The main dataset limitations are typical for fake news corpora: source-domain bias, topic imbalance, and the possibility that label quality depends on historical fact-checking context

rather than intrinsic linguistic quality alone. Those limitations matter because a strong classifier may still generalize poorly when content style or topic mix changes. For that reason, the final system includes monitoring and a dedicated robustness discussion instead of claiming permanent or universal reliability.

5 Data Processing

The preprocessing pipeline is implemented in `src/data/preprocessing.py` and applies multiple deterministic cleaning steps: HTML removal, URL removal, lowercasing, special-character handling, and whitespace normalization. The design choice here is pragmatic. The baseline model benefits strongly from normalization because TF-IDF features are sensitive to superficial noise, while the transformer still receives cleaner input without losing the semantic structure of the article.

The cleaning pipeline is applied through typed configuration rather than hard-coded preprocessing behavior. This is important because it allows preprocessing decisions to remain explicit and reproducible. The code also drops empty texts, normalizes labels, and removes duplicate rows by text-label pair, which reduces evaluation distortion due to repeated examples.

The split pipeline is implemented in `src/data/splitter.py`. It first validates that split ratios sum to one, then creates a held-out test set and a train/validation split inside the remaining data. Stratification is used when the label distribution supports it. This two-stage design is simple, auditable, and appropriate for a course project that must emphasize correct experimental handling.

A production-minded system also needs to consider missing, noisy, and potentially biased data. The code already guards against missing labels and empty text after preprocessing. At a higher level, the project treats robustness and bias as explicit documentation and monitoring concerns because a clean split is necessary but not sufficient for trustworthy deployment.

6 Baseline Model (TF-IDF + Logistic Regression)

The baseline model is configured in `configs/training.yaml` and trained through `src/training/train_baseline.py`. It uses TF-IDF with up to 20,000 features, unigrams and bigrams, document-frequency thresholds, and sublinear term frequency. Logistic Regression uses a balanced class weighting strategy and a solver suitable for sparse linear classification.

This baseline is not included merely for formality. It performs two important roles. First, it provides a transparent benchmark that verifies the quality of the dataset and preprocessing pipeline. Second, it provides a concrete comparison for the course requirement that the final model be evaluated against at least one simpler alternative.

The recorded evaluation metrics show that the baseline is already very strong. Its accuracy is approximately 0.9913 and its F1 is approximately 0.9904. This means the dataset carries strong lexical and phrase-level cues that a linear classifier can exploit effectively. That result strengthens the final report because it shows the transformer must outperform a non-trivial benchmark rather than an intentionally weak strawman.

Despite its strengths, the baseline has known limitations. It cannot model long-range compositional meaning as effectively as a transformer, and it is more brittle under paraphrase or contextual framing changes. Those constraints justify the transition to RoBERTa while still preserving the baseline as a useful operational and scientific reference point.

7 Transformer Model (RoBERTa)

The main model is `roberta-base`, configured in `configs/model.yaml` and trained through `src/training/train_transformer.py`. The training setup uses maximum sequence length 512, batch size 16, learning rate 2×10^{-5} , three epochs, weight decay 0.01, warmup ratio 0.1, and selection of the best checkpoint by validation F1. These choices are consistent with standard fine-tuning practice for transformer text classification.

The predictor used for inference is implemented in `src/inference/predictor.py`. It loads the tokenizer and sequence-classification model from the saved model directory and exposes a compact `predict()` interface returning label and confidence. This separation between training and inference is important for deployment, because the backend imports a predictor abstraction instead of embedding training logic into the serving path.

The transformer training entry point includes additional engineering behavior that reflects real-world constraints. It can discover the latest checkpoint, resume interrupted training, print diagnostics for environment debugging, and export comparison artifacts against the baseline. These are practical details that matter when moving from an experiment to a repeatable system.

RoBERTa is justified not only by its final score but by its ability to capture richer context. Fake news claims often depend on tone, negation, syntactic framing, or combinations of entities and events that are poorly represented by sparse linear features. The model therefore fits the problem better when the goal is robust deployed inference rather than only leaderboard performance.

8 Agentic AI Architecture

The agentic component is centered on `src/agent/controller.py`, which orchestrates classification, evidence retrieval, decision synthesis, explainability generation, metrics recording, and trace persistence. This directly satisfies the course requirement that the project contain multi-step reasoning and decision-making based on intermediate outputs. The workflow is not a static single-model call; it is a sequence of tools with explicit intermediate state.

The agent controller first classifies the article, then retrieves evidence, then combines those signals in a decision tool, and finally requests an explanation from the explainability service. Every step can be recorded in an execution trace. The final user-visible output contains not only prediction and confidence but also trust score, human review state, conflict flag, evidence summary, source information, and trace data.

The architecture is intentionally lightweight. Instead of using a large external orchestration framework for every action, the project implements a clear, testable controller around internally defined tools. This keeps the system maintainable while still exhibiting genuine agentic behavior. The design also respects the stated project constraint of not retraining or replacing the underlying RoBERTa checkpoint during the Day 1 to Day 4 evolution.

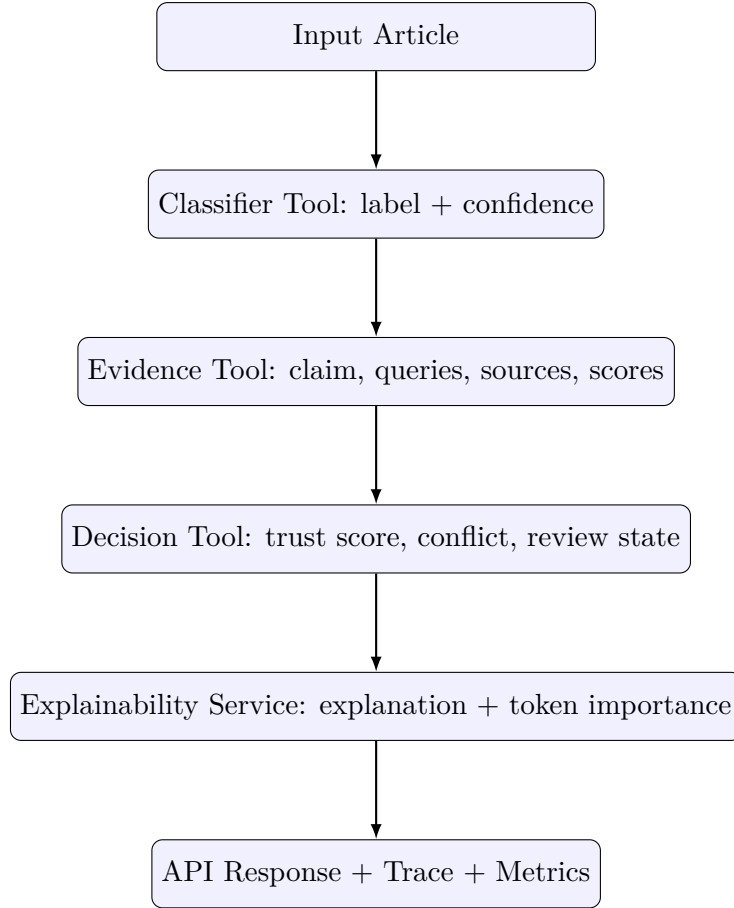


Figure 1: Agentic workflow implemented by the project controller.

9 Evidence Retrieval Pipeline

The evidence retrieval pipeline is implemented through `src/agent/evidence_tool.py` and the retrieval modules under `src/retrieval/` and `src/agents/retrieval/`. The tool extracts a claim, generates or reuses search queries, collects evidence sources, scores alignment, estimates source credibility, and summarizes whether the evidence supports or contradicts the article.

A notable design choice is the use of HTML-based fallback retrieval behavior instead of depending exclusively on a single external API key. The retrieval path can use a hybrid search client with Wikipedia and Bing HTML search clients, which improves robustness for academic demonstration. This matters because deployment readiness is not only about model serving; it also depends on whether the system can degrade gracefully when an external dependency is unavailable.

The pipeline assigns per-source fields including title, source, URL, relevance, provider, source credibility, stance, and matched terms. Those fields are then aggregated into support score, contradiction score, evidence quality score, and conflict flag. This makes the evidence layer computationally simple but operationally useful, because downstream components can reason over structured evidence rather than raw snippets only.

The current evidence scoring is heuristic, and the project explicitly documents that limitation. That is acceptable within the course scope because the requirement is to include tool usage and decision-making based on intermediate outputs, not to solve every retrieval or stance-analysis problem optimally. The important achievement is that the final system can operationalize evidence as a first-class signal.

10 Explainability System

The explainability system combines prediction context, evidence context, and token importance into a final human-readable explanation. It is called from the agent controller after the decision phase and returns both explanation text and important tokens. The frontend then renders these outputs together with trust score and evidence cards.

A key architectural correction made during the final stabilization was to treat the trust score coming from the decision layer as the authoritative value rather than recomputing it inside the explanation formatter. This avoids inconsistency between the decision module and the user-facing explanation. In product terms, this matters because conflicting trust values would reduce user confidence in the system more than a slightly lower raw score would.

The final explanation also includes evidence support, contradiction, source credibility, and human review state. As a result, the explanation is not limited to word attribution. It communicates the reasoning context around the prediction, which is much closer to how a reviewer would want to understand a suspicious article. This design therefore supports the rubric’s requirement for explainability considerations for non-technical stakeholders.

The explainability layer remains intentionally lightweight and rule-compatible with the existing model. It does not require retraining the checkpoint and therefore respects the project constraint that the classifier remain unchanged while orchestration and presentation improve around it.

11 Evaluation Methodology

Evaluation is performed on a held-out test split created through the data splitting pipeline. The report uses accuracy, precision, recall, and F1 as primary classification metrics. These are complemented by confusion matrix inspection and error analysis artifacts stored in `artifacts/evaluation/`.

The methodology also includes direct baseline comparison. This is important because a high transformer score alone does not demonstrate good experimental practice. The course rubric requires at least one baseline and a discussion of why the final model is preferable. By training and evaluating both approaches on the same split logic, the repository satisfies that requirement in a reproducible way.

For the agentic layer, evaluation is broader than plain classification metrics. The project tests evidence retrieval, conflict handling, the UNCERTAIN state, trace serialization, and endpoint behavior. The agent endpoint tests in `tests/api/test_agent_endpoint.py` verify real, fake, and uncertain outcomes and ensure trace data is returned. This is an important methodological point because user trust depends on orchestration behavior, not only on classifier output.

Finally, operational methodology includes health and version endpoints, logging, monitoring artifacts, and container configuration. These elements do not replace classification evaluation, but they do satisfy the deployment and lifecycle requirements of the assignment.

12 Experimental Results

The evaluation artifacts record strong performance for both the baseline and the transformer model. Table ?? summarizes the final comparison using the generated artifact values.

Table 1: Baseline versus RoBERTa evaluation results

Model	Accuracy	Precision	Recall	F1
TF-IDF + Logistic Regression	0.9913	0.9938	0.9870	0.9904
RoBERTa	0.9991	0.9987	0.9997	0.9992

The transformer outperforms the baseline on all reported metrics. The absolute improvement is modest because the baseline is already extremely strong, but the transformer still yields better recall and slightly better balance across errors. In a fake news setting, higher recall is particularly valuable because it reduces the chance of missing problematic misinformation instances.

The confusion matrix in the saved metrics artifact is:

$$\begin{bmatrix} 2605 & 4 \\ 1 & 3137 \end{bmatrix}$$

This result indicates only five misclassifications in the held-out evaluation artifact. The remaining errors are therefore rare, which shifts the focus of system design from raw model accuracy toward explainability, evidence quality, and operational trust.

Screenshot reference: `docs/slides/images/confusion_matrix.png`

Figure 2: Confusion matrix reference used in the presentation assets and consistent with saved evaluation artifacts.

The project also includes error analysis CSV output, which supports post-hoc inspection of difficult cases. While the model accuracy is very high, the report does not claim perfection. Instead, it emphasizes that ambiguous or mixed-context articles remain failure points and should be surfaced to human review when evidence signals are weak or conflicting.

13 Deployment Architecture

The deployed system consists of a Streamlit frontend, a FastAPI backend, the RoBERTa inference service embedded through the predictor abstraction, an evidence retrieval path, and file-based monitoring artifacts. The Dockerfile builds a slim Python container with all application dependencies, while `docker-compose.yml` defines separate services for the API and frontend with health checks.

The backend is structured around `src/api/main.py`, `src/api/routes.py`, and `src/api/health.py`. Middleware records request IDs and latency, routes validate input, and the health and version

endpoints support deployment probes. The frontend connects to the backend over HTTP and renders results in a way suitable for demonstration, screenshots, and defense discussion.

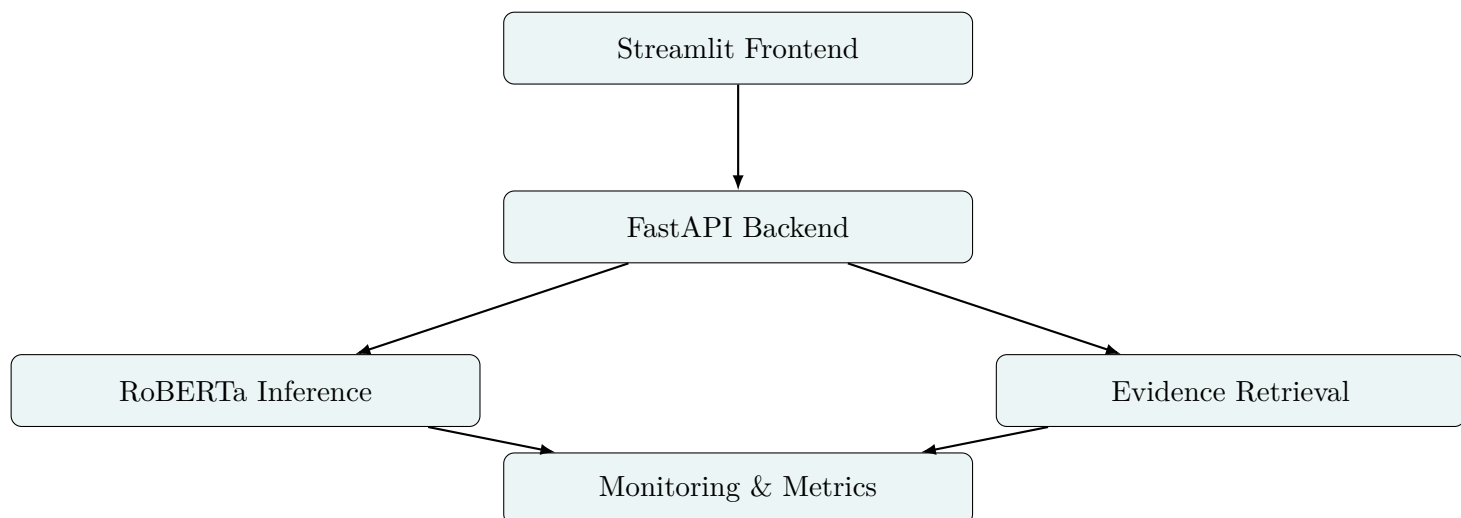


Figure 3: Deployment architecture for the final submission system.

This architecture is appropriate for the assignment because it demonstrates deployability without overcomplicating infrastructure. It can run locally, in Docker, or conceptually on cloud services such as Render and Railway. At the same time, the report explicitly recognizes that production-grade public deployment would still require stronger logging backends, request throttling, and external secret management.

14 Ethics, Privacy, and Limitations

The system addresses privacy by masking personally identifiable information before inference and logging. The masking utility replaces detected entities such as emails, phone numbers, URLs, and addresses with stable placeholders. This design reduces the risk of storing raw sensitive text in downstream systems and satisfies the rubric’s requirement for privacy analysis and minimization strategies.

Ethically, fake news detection systems carry risks of false positives, false negatives, and trust overreach. A false positive can suppress legitimate information, while a false negative can allow harmful misinformation to spread. The project mitigates this by introducing an operational UNCERTAIN state, conflict flags, and evidence-based explanation rather than presenting every output as a fully certain verdict.

Robustness remains a significant limitation. Evidence retrieval depends on public web availability, stance analysis is heuristic, and domain shifts could degrade both model and retrieval performance. The project does not hide these limitations; instead, it records them in documentation, monitoring design, and defense materials. This is an important part of responsible AI practice.

Another limitation is that the current monitoring stack is file-based and academic-demo oriented. It supports drift and confidence tracking, but it is not a full observability platform. Likewise, the current system does not perform automated retraining in production. These limitations are acceptable within scope because the rubric allows conceptual continual learning design even when a full implementation is not feasible.

15 Future Work

The highest-value technical improvement is stronger evidence alignment. A future version should replace heuristic support/contradiction detection with a dedicated stance model or retrieval-augmented reasoning pipeline. This would improve the quality of the trust score and reduce the number of cases where evidence is retrieved but poorly interpreted.

A second improvement is broader robustness and domain coverage. The classifier is strong on the evaluated distribution, but future work should evaluate out-of-domain generalization more aggressively, including multilingual content, platform-native short claims, and adversarial paraphrases. Additional fairness and subgroup analysis would also improve the responsible AI profile of the system.

A third improvement is operational hardening. The current Docker-based deployment is sufficient for grading and demonstration, but public deployment would benefit from centralized logs, rate limiting, managed secrets, CI-based image publishing, and cloud-native scaling policy. These are natural next steps for moving from academic prototype to real-world service.

Finally, the frontend could evolve from a demonstration UI into a reviewer workflow dashboard with case history, saved traces, and reviewer feedback collection. That would also strengthen the continual learning loop by creating an explicit path for new labeled corrections.

16 Conclusion

This project satisfies the course objective of building a realistic end-to-end NLP system rather than an isolated modeling experiment. It includes data handling, a baseline model, a transformer model, a deployable backend and frontend, an agentic orchestration layer, monitoring concepts, privacy handling, and defense-ready documentation. The result is both technically strong and architecturally coherent.

RoBERTa provides the best classification performance, but the most important final submission contribution is the surrounding system design. Evidence retrieval, trust scoring, explanation, conflict detection, trace capture, and operational endpoints transform the classifier into a more reviewable decision-support tool. This is what makes the project aligned with production-oriented NLP practice.

The final verdict is therefore not only that the model performs well, but that the repository now demonstrates the full lifecycle expected by Prokct-2: problem framing, structured implementation, evaluation, deployment, agentic behavior, monitoring, and responsible AI reflection.

A API Workflow Summary

The API layer exposes `/predict`, `/analyze`, `/agent`, `/health`, and `/version`. The `/predict` route returns direct classifier output, `/analyze` preserves the legacy full workflow, and `/agent` returns the newer production-facing agentic orchestration result with trust and trace metadata. This split is architecturally useful because it supports both compatibility and final defense demonstration.

A.1 Endpoint Contract Table

Endpoint	Method	Purpose	Main Output
<code>/health</code>	GET	Deployment liveness probe	status=healthy
<code>/version</code>	GET	Model/version metadata	model name and version string
<code>/predict</code>	POST	Direct classifier inference	prediction label and confidence
<code>/analyze</code>	POST	Legacy end-to-end workflow	prediction, evidence, explanation details
<code>/agent</code>	POST	Final agentic workflow	prediction, trust score, review state, sources, trace

A.2 Response Design Notes

The existence of both `/analyze` and `/agent` is intentional. The former preserves compatibility with the earlier multi-agent workflow surface, while the latter is optimized for the final defense and frontend integration. This separation lowered migration risk late in the project and made it possible to stabilize the UI against a single canonical response shape. The practical output difference is that `/agent` foregrounds trust, evidence quality, human review state, and trace data, all of which are central to the final defense narrative.

B Agent Decision Logic Summary

The decision layer translates intermediate signals into a final operational recommendation. This matters because a high-confidence classifier can still behave unsafely when external evidence is absent or contradictory. The final project therefore uses a decision synthesis step instead of treating confidence as a terminal answer.

Table 3: Decision-layer interpretation of intermediate signals

Condition	Output Tendency	Operational Meaning
No evidence found	Lower trust, UNCERTAIN	Classifier alone is insufficient for strong review recommendation
Evidence conflicts with classifier	Conflict flag, UNCERTAIN	Human validation required before action
Weak evidence quality	Lower trust, UNCERTAIN	Retrieved sources are not strong enough to justify certainty
Evidence aligns with classifier	REAL or FAKE	Final decision can be surfaced with stronger confidence
Low classifier confidence	UNCERTAIN or medium-risk result	Model uncertainty should remain visible to the user

This logic gives the final system a safer interaction model. Rather than over-asserting every output, it explicitly surfaces uncertainty and conflict, which is more defensible in both academic and operational settings. The project therefore treats the classifier as one tool inside a larger decision process rather than the entire system.

C Monitoring and Lifecycle Metrics

The monitoring layer is part of the final submission because the rubric explicitly asks for continual learning and monitoring design. The implemented stack records prediction-side behavior, confidence trends, model health, and agent metrics. Although fully automated retraining is not enabled, the observability layer is concrete enough to support post-deployment review and conceptual lifecycle management.

Table 4: Monitoring metrics and their operational role

Metric	Source	Why It Matters
Prediction confidence trend	confidence monitor	Detects model uncertainty drift over time
Prediction distribution drift	prediction drift monitor	Reveals label skew changes in production traffic
Data drift score	data drift monitor	Captures shifts in textual input characteristics
Agent conflict rate	agent metrics tracker	Indicates disagreement between classifier and evidence
UNCERTAIN rate	agent metrics tracker	Measures how often the system escalates to human review
Average response time	request middleware and agent metrics	Supports latency and deployment analysis
Evidence retrieval success rate	agent metrics tracker	Evaluates the reliability of retrieval-dependent reasoning

From a lifecycle perspective, these metrics are intended to support threshold-based retraining review. For example, a rising uncertainty rate combined with falling evidence retrieval success could suggest either domain shift or retrieval degradation. Similarly, sustained confidence deterioration may indicate data drift even when top-line latency remains acceptable. This is why the final monitoring design emphasizes multiple signals rather than one summary score.

D Deployment Environment Comparison

The repository is structured for local deployment, Dockerized single-host deployment, and straightforward migration to hosted services. The project does not claim that all production hardening is complete, but it does show a credible path from local demo to cloud-hosted service.

Table 5: Deployment environment comparison

Environment	Strength	Limitation	Best Use
Local machine	Fastest iteration cycle	Manual setup and local dependency risk	Development and defense rehearsal
Docker Compose	Reproducible service boundary	Requires Docker availability	Academic demonstration and grading
Render or Railway	Easy hosted deployment	Limited free-tier resources	Public demo or lightweight staging
VPS or cloud VM	Full control over runtime	Higher operational burden	Advanced extension beyond course scope

The health endpoint, version endpoint, and two-service Docker Compose layout were selected because they represent the minimum credible deployment story for an NLP system in an academic context. This is also why the project documentation distinguishes demo readiness from public production readiness. The final submission is deployment-minded and reproducible, but it does not misrepresent itself as a fully hardened public platform.

E UAT Coverage Summary

The final user acceptance package covers not only positive prediction flows but also safety and operational edge cases. This is important because an end-user system can fail even when its classifier accuracy is high. The UAT design therefore checks empty input handling, backend unavailability, trace rendering, evidence cards, monitoring hooks, and route-level behavior.

Test ID	Scenario	Expected Behavior	Validation Basis
UAT-01	Health probe	API returns healthy status	Localhost runtime fetch
UAT-04	Empty input in UI	User sees validation message	Streamlit code path review
UAT-08	Weak evidence case	Review state becomes UNCERTAIN	Agent endpoint tests
UAT-09	Conflict case	Conflict flag is true	Agent endpoint tests
UAT-13	PII-containing input	Text is masked before inference	Route layer and privacy module
UAT-18	Evidence card rendering	Provider, URL, trust, and preview display	Frontend rendering logic
UAT-19	Trace timeline rendering	Steps and durations are shown	Frontend trace rendering logic
UAT-20	Monitoring during predict	Logging must not block responses	Route try/except monitoring hooks
UAT-22	Validation failure	Structured API error response	FastAPI exception handlers
UAT-24	Startup commands	README commands match active services	Documentation and runtime probes

This UAT summary reinforces a core submission claim: the project is not only about model quality but also about runtime behavior, fallback handling, and user-facing correctness.

F Project Evolution from Day 1 to Day 4

The Day 1 to Day 4 progression is important because it explains how the project became more product-like without destabilizing the model checkpoint. Day 1 established agent tools and architectural framing. Day 2 introduced the controller and integration behavior. Day 3 made the system evidence-aware. Day 4 hardened the workflow with monitoring, conflict handling, and defense artifacts.

Table 7: High-level evolution of the agentic system

Phase	Primary Change	Submission Impact
Day 1	Tool and architecture foundation	Defined the agentic direction and reusable components
Day 2	Controller integration and test expansion	Connected agent behavior to the backend and API surface
Day 3	Evidence-aware routing and validation	Added external-grounding logic and uncertainty handling
Day 4	Hardening, traces, metrics, and documentation	Made the system defense-ready and easier to audit

This evolution path also explains a major design decision: the project did not retrain or replace the RoBERTa checkpoint during the final migration. Instead, it improved orchestration, transparency, and operational quality around a stable classifier. That was the correct engineering trade-off for the submission timeline because it reduced late-stage model risk while still raising overall system quality.

G Configuration Snapshot

The repository uses YAML-based configuration to keep training, dataset handling, evaluation, retrieval, and monitoring behavior explicit. This is a practical engineering choice because it reduces hidden assumptions and makes it possible to reason about experiments and deployment settings separately.

Table 8: Baseline configuration snapshot

Parameter	Value
Experiment name	<code>fake_news_baseline</code>
TF-IDF max features	20,000
N-gram range	unigram and bigram
Minimum document frequency	2
Maximum document frequency	0.95
Logistic Regression solver	<code>liblinear</code>
Maximum iterations	1000
Class weighting	balanced

Table 9: Transformer configuration snapshot

Parameter	Value
Model name	<code>roberta-base</code>
Maximum sequence length	512
Batch size	16
Learning rate	2×10^{-5}
Epochs	3
Weight decay	0.01
Warmup ratio	0.1
Best model selection metric	F1

These tables make the report more auditable during defense. An examiner can directly connect the report claims to repository configuration files rather than relying on prose alone. That is especially valuable for reproducibility questions.

H Failure Modes and Mitigation Matrix

No responsible misinformation system should claim perfect reliability. The final repository therefore treats failure modes as an explicit engineering concern. The table below summarizes the most important failure classes and their current mitigation strategy.

Failure Mode	Example	Severity	Current Mitigation
Classifier overconfidence	Strong lexical cue but wrong factual implication	High	Evidence retrieval, trust score, UNCERTAIN state
Weak or unavailable evidence	Public sources unavailable or irrelevant	Medium to high	Fallback retrieval clients, lower trust, human review
Conflicting evidence	Sources disagree with classifier output	High	Conflict flag and UNCERTAIN review state

Failure Mode	Example	Severity	Current Mitigation
Privacy exposure risk	Input contains email or phone number	High	PII masking before inference/logging
Domain shift	New topic or style not seen in training	Medium	Monitoring, drift detection, conceptual re-training plan
UI misunderstanding	User reads confidence as factual truth	Medium	Separate trust score, explanation text, evidence display
Latency spike	Retrieval path becomes slow	Medium	Independent health probes, metrics, deployment guidance

The practical value of this matrix is that it links the technical design to operational responsibility. The system does not merely state that limitations exist; it maps them to explicit interface or backend behaviors.

I Team-Style Planning and Task Allocation

Although the repository was developed in a course setting, the final submission also reflects team-style planning. This matters because the rubric asks for project management and teamwork evidence, even in solo or semi-simulated form. The project phases can be interpreted as coordinated workstreams across modeling, backend, frontend, DevOps, QA, and technical writing.

Table 11: Simulated team-role allocation

Role	Responsibility
ML Engineer	Baseline training, RoBERTa fine-tuning, metrics interpretation
Data Engineer	Data preparation, preprocessing, split reproducibility, validation
Backend Engineer	FastAPI routes, schemas, dependency wiring, error handling
Frontend Engineer	Streamlit UX, evidence display, trace timeline, screenshot mode
QA Engineer	Endpoint tests, frontend helper tests, UAT coverage, regression checks
DevOps Engineer	Docker assets, health checks, deployment instructions, environment design
Technical Writer	Report, slides, defense package, requirements matrix, readiness docs

This view is not artificial padding. It makes explicit the actual engineering concerns represented in the repository and provides a concrete answer when an examiner asks how the project would scale in a real team environment.

J Defense Demo Script Summary

The defense demonstration should highlight the system as an integrated product rather than a sequence of disconnected files. A strong demo path is therefore: start from the Streamlit UI, show a factual article example, show a suspicious or fabricated example, then explain how evidence, trust score, and trace differ between those cases.

An effective demo should include at least one scenario where the classifier and evidence align and one where the final system escalates to UNCERTAIN. That contrast is critical because it shows why the agentic layer exists. Without it, the defense risks collapsing into a generic text-classification demonstration.

The demo should also verify two operational endpoints live: `/health` and `/version`. This is useful because it immediately communicates deployment readiness and shows that the system is not only a static UI mockup. Finally, if time allows, the examiner should be shown the trace or evidence cards for at least one example to reinforce transparency claims.

K Release Checklist and Hosting Cost Snapshot

The final submission process also benefits from an explicit release checklist because this project contains multiple artifact types: code, report, slides, Docker assets, evaluation outputs, and supporting documentation. A release-oriented checklist reduces the risk of inconsistent packaging late in the submission process.

Table 12: Final release checklist snapshot

Checklist Item	Status	Expecta- tion
FastAPI health endpoint responds	Required	
Streamlit homepage loads	Required	
Canonical report PDF exists	Required	
Canonical slide PDF exists	Required	
README startup instructions are current	Required	
Final requirements matrix is present	Required	
UAT, deployment, and defense docs are present	Required	
No secrets or transient build noise staged	Required	

The deployment-readiness review also considered approximate hosting cost. A small Render or Railway deployment is usually enough for a demonstration-grade service, typically in the range of five to twenty-five USD per month depending on traffic and compute needs. A small VPS gives more control at the cost of more manual operations, while a cloud VM on a major provider increases flexibility but also operational overhead. These estimates are useful in defense because they connect architecture choices to practical deployment trade-offs.

L Frontend Screenshots



Figure 4: Streamlit frontend screenshot reference used for demonstration and report integration.

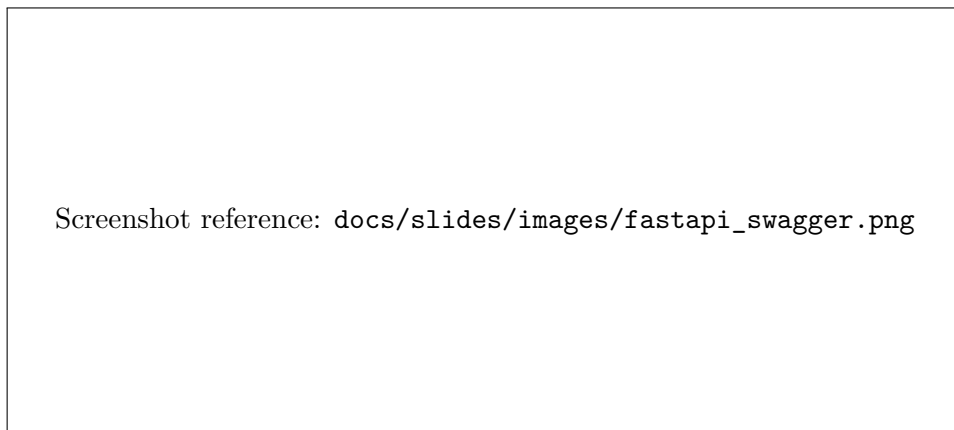


Figure 5: FastAPI Swagger reference showing the deployment interface.

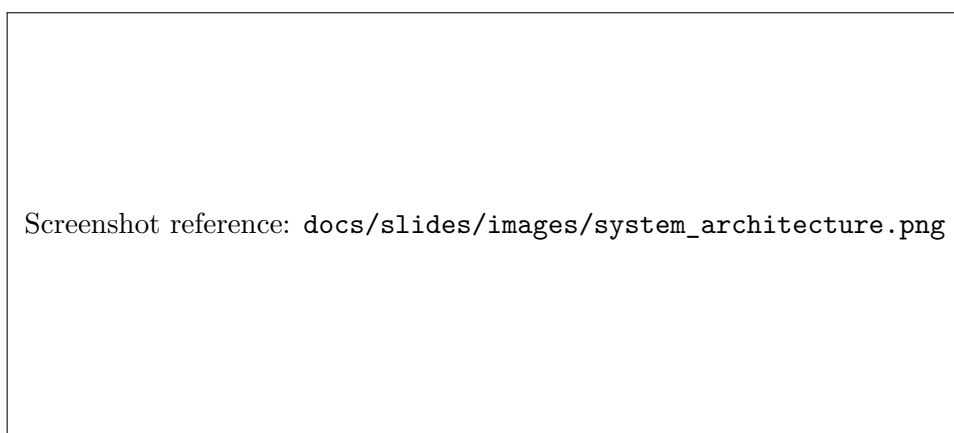


Figure 6: System architecture reference used in presentation assets.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *NAACL-HLT*, 2019.

- [2] Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv:1907.11692, 2019.
- [3] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, “Fake News Detection on Social Media: A Data Mining Perspective,” *ACM SIGKDD Explorations Newsletter*, 2017.
- [4] S. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *NeurIPS*, 2017.
- [5] A. Vaswani et al., “Attention Is All You Need,” in *NeurIPS*, 2017.